



GUIA DEFINITIVO & PLANEJAMENTO DA A3: MINILANG

Compilador Completo para uma Linguagem Imperativa | Disciplina Teoria da Computação e Compiladores

INSTITUIÇÃO & DOCENTE
UNIFACS | Prof. Daniel
Santana

EQUIPE DE ALUNOS
2 a 4 Alunos (Alinhado em
Sala)

AValiação & PONTOS
A3 Semestral — 40 Pontos

SEMESTRE & CARGA
2026.2 — 160 Horas

1. Visão Geral: O que o Professor Espera

A A3 tem como objetivo construir um **compilador ou interpretador completo** para uma linguagem imperativa chamada **MiniLang**. Diferente de matérias convencionais, a entrega é **cumulativa e dividida em 4 marcos (M1 a M4)** com avaliação contínua. Não existe "entrega única na véspera".

🔧 Produtos Finais Avaliados pelo Professor:

- **Código-fonte em Repositório Git**
com histórico contínuo de commits de todos os membros da equipe (commit único gera penalização).
- **Pipeline Funcional em Linha Única**
documentado no `README.md` (ex: `python minilang.py programa.ml`).
- **Suíte de Testes**
com bateria de programas válidos e casos inválidos (testando mensagens de erro).
- **Relatório Técnico Acadêmico**
de 6 a 10 páginas (detalhando decisões de projeto, EBNF, AFD e limitações).
- **Apresentação de 15 Minutos**
com demonstração ao vivo e
arguição individual de cada integrante

2. Especificação Oficial da Linguagem MiniLang

A MiniLang é uma linguagem imperativa com comandos de entrada/saída, controle de fluxo e expressões.

Elemento	Regra e Especificação Oficial
Palavras Reservadas (15)	<code>programa</code> , <code>var</code> , <code>inteiro</code> , <code>booleano</code> , <code>se</code> , <code>senão</code> , <code>enquanto</code> , <code>escreva</code> , <code>leia</code> , <code>verdadeiro</code> , <code>falso</code> , <code>e</code> , <code>ou</code> , <code>não</code> , <code>fim</code>
Identificadores	Iniciados por letra, seguidos de letras, dígitos ou sublinhado <code>_</code> . Proibido colidir com palavras reservadas.
Tipos Primitivos	<code>inteiro</code> (ex: <code>42</code>) e <code>booleano</code> (<code>verdadeiro</code> , <code>falso</code>).
Operadores	• Aritméticos: <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> • Relacionais: <code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> • Lógicos: <code>e</code> , <code>ou</code> , <code>não</code> Atribuição: <code>=</code>
Delimitadores	Parênteses <code>()</code> , Chaves <code>{ }</code> , Ponto e vírgula <code>;</code> , Dois-pontos <code>:</code> , Vírgula <code>,</code> e Ponto final <code>.</code>
Comentários	Caractere <code>#</code> até o fim da linha (comentário de linha única, descartado pelo analisador léxico).

Elemento	Regra e Especificação Oficial
Tratamento de Erros	Obrigatório em todas as fases: relatar fase (Léxico, Sintático, Semântico), linha e coluna exatas com mensagem explicativa.

3. Marcos de Avaliação, Prazos e Rubrica de Pontos (40 Pontos)

Marco 1: Analisador Léxico

8 Pontos | ~21/09 (Semana 7)

- Reconhecer todas as categorias de tokens (3,0 pts)
- Tratar lookahead: `=` / `==` , `<` / `<=` , `>` / `>=` , `!=` (1,0 pt)
- Descartar comentários `#` e espaços com linha/coluna (1,0 pt)
- Reportar erros léxicos com posição exata (1,0 pt)
- Documentar formalmente o AFD** (diagrama ou tabela) (1,5 pts)
- Suíte de testes válidos e inválidos (0,5 pt)

Marco 2: Analisador Sintático + AST

12 Pontos | ~12/10 (Semana 10)

- Aceitar programas válidos da bateria (3,0 pts)
- Rejeitar inválidos com erro e linha/coluna corretas (2,0 pts)
- Recuperação em Modo Pânico** (múltiplos erros por run) (1,5 pts)
- Construir AST navegável** (não só aceita/rejeita) (3,0 pts)
- Precedência e associatividade de operadores (1,5 pts)
- Resolver e documentar o Dangling Else** (1,0 pt)

Marco 3: Analisador Semântico

10 Pontos | ~09/11 (Semana 14)

- Tabela de símbolos com escopo, tipo e posição (3,0 pts)
- Detectar var não declarada, redeclaração e tipos incompatíveis (3,0 pts)
- Validar operadores corretos e `leia` em var declarada (2,0 pts)
- Anotar AST com os tipos inferidos (1,5 pts)
- Bônus:* Detectar var lida antes de inicializada (+0,5 pt)

Marco 4: Back-End + Relatório + Demo

10 Pontos | ~30/11 (Semana 17)

- Back-end funcional: TAC ou Interpretador da AST (4,0 pts)
- Demonstrar ao menos 1 otimização (antes vs depois) (1,5 pts)
- Relatório técnico de 6 a 10 páginas (2,5 pts)
- Apresentação de 15 min com demo ao vivo (1,5 pts)
- Domínio técnico individual sob arguição (0,5 pt)

4. Extensão Obrigatória: Regra Vital de Nota

⚠️ ATENÇÃO CRÍTICA AO TETO DE NOTA:

O documento especifica que equipes com 3 ou mais integrantes (e formalmente qualquer equipe do projeto padrão) **devem escolher 1 extensão no início** e implementá-la até o M4.

Sem a extensão implementada, a nota máxima do Marco 4 é limitada a 7,0 de 10!

Opção	Extensão	O que exige a mais no compilador	Avaliação do Supervisor
A	Procedimentos	Procedimentos sem retorno, parâmetros por valor. Exige escopo aninhado na tabela de símbolos e pilha de ativação.	Complexidade Alta (Back-end complexo)
B	Vetores	Vetores 1D de inteiros. Exige tipo composto, cálculo de deslocamento de memória e checagem de limites de índice.	Complexidade Média-Alta
C	Tipo Real	Adicionar tipo <code>real</code> com coerção/promoção automática de <code>inteiro</code> para <code>real</code> .	Complexidade Média (Afeta checagem de tipos)
D	Comandos Para e Repita/Até	Comandos <code>para</code> e <code>repita ... até</code> . Tratados como <i>açúcar sintático</i> (*syntactic sugar*), traduzidos na própria AST para <code>enquanto</code> .	RECOMENDADA (Melhor Custo/Benefício)
E	Cadeias de Caracteres	Tipo texto (string) com literais <code>"..."</code> e concatenação. Exige novo token e suporte a strings no interpretador.	RECOMENDADA (Alternativa Viável)

5. Recomendações Estratégicas do Supervisor para Nota Máxima

1. Stack de Desenvolvimento: Python 3

Por que Python? Em Compiladores acadêmicos, Python elimina dores de cabeça com ponteiros de C/C++ ou verbosidade de Java. Permite criar classes de nós da AST em `dataclasses` e escrever o interpretador com o padrão *Visitor* em poucos arquivos legíveis. Isso garante que **todos os membros da equipe consigam ler, entender e explicar o código na arguição.**

2. Estratégia de Parsing: Descendente Recursivo

Em vez de geradores automáticos complexos (como Bison ou ANTLR) que geram milhares de linhas de código indecifrável, um **Recursive Descent Parser manual** é 100% didático. Cada regra da gramática vira uma função simples (ex: `parse_comando()`, `parse_expressao()`), sendo fácil de debugar e perfeito para recuperar erros no modo pânico.

3. Escolha da Extensão: Opção D (Para / Repita)

A Opção D é a escolha mais inteligente: o comando `para (i = 0; i < 10; i = i + 1)` pode ser transformado (desaçuçarado) durante o parsing diretamente em uma atribuição inicial seguida de um nó `EnquantoNode` com incremento ao final! Isso significa que **o semântico e o interpretador não precisam de instruções novas complexas.**

4. Otimização do M4: Constant Folding

Para cumprir o requisito de otimização do M4 (1,5 pts), o mais robusto e elegante é implementar o **Dobramento de Constantes (Constant Folding)**. Uma passada na AST calcula operações estáticas antes da execução (ex: `x = 10 + 2 * 5`; vira `x = 20`; na árvore). Fácil de demonstrar no relatório com "antes e depois".

6. Governança de Git e Divisão de Papéis na Equipe (2 a 4 Alunos)

O professor frisou: "O histórico de commits conta como evidência de processo; commit único na véspera será penalizado."

Sugestão de Divisão de Responsabilidades (Exemplo para 3-4 Integrantes):

- **Membro 1 (Léxico, Tokens & Testes):**
Scanner, tabela de transições/AFD, tracking de linha/coluna, bateria de testes do M1.
- **Membro 2 (Gramática, Parser & AST):**
Gramática EBNF, funções do parser descendente recursivo, resolução do Dangling Else.
- **Membro 3 (Tabela de Símbolos & Semântica):**
Escopos, verificação de tipos, checagem de variáveis e anotação da AST.
- **Membro 4 (Back-End, Otimização & Relatório):**
Interpretador da AST, Constant Folding, documentação e slides.

* Se a equipe tiver 2 integrantes, divide-se em: Front-end (Léxico/Sintático) e Back-end (Semântico/Execução/Docs).

7. Checklist Operacional Imediato para o Marco 1 (M1)

- [] **Repositório Git Criado:** Inicializar repositório no GitHub/GitLab e adicionar o e-mail/usuário do professor.
- [] **Definição da Stack:** Fechar em equipe o uso de Python 3 com estrutura de pastas modular (`src/` , `tests/` , `docs/`).
- [] **Especificação do AFD:** Desenhar o diagrama do autômato finito com todos os estados para os tokens e operadores.
- [] **Implementação do Scanner:** Construir a classe `Lexer` reconhecendo os 15 identificadores, operadores com lookahead e comentários `#` .
- [] **Testes Automatizados:** Criar pasta de testes com 5 exemplos de código válidos e 5 com erros léxicos provocados.
- [] **Nota Técnica do M1:** Elaborar o relatório de 1 página contendo a imagem do AFD e instruções de execução.